Residency Companion
Technical Proposal

AI-Powered Medical Decision Support for India

Contents

1 Introduction **2**

2 Understanding the Vision: *Residency Companion* as an *OpenEvidence* Competitor **2**
2.1 Key Functions of *OpenEvidence* . . . 2
2.2 How *OpenEvidence* Works (Technical Components) . . . 2
2.3 Challenges Adaptations for India . . . 3
2.3.1 Local Content & Guidelines: . . . 3
2.3.2 Credential Verification: . . . 3
2.3.3 Technical Infrastructure: . . . 3
2.3.4 Feature Set: . . . 3
2.3.5 Problem-Solution Mapping for India: . . . 3
2.4 Sourcing Training Material (Indian Context) . . . 3

3 Development Plan: *Residency Companion* Technical Roadmap **4**
3.1 Core Technology Choices . . . 4
3.2 Phase 1: Foundation & Content Ingestion System (MVP) . . . 4
3.2.1 Objective: . . . 4
3.3 Phase 2: Knowledge Base Creation & AI Integration . . . 5
3.3.1 Objective: . . . 5
3.4 Phase 3: Multi-Channel User Interfaces & Enhancements . . . 5
3.4.1 Objective: . . . 5
3.5 Phase 4: Advanced Features Optimization . . . 6
3.5.1 Objective: . . . 6

4 Technology Stack Summary **7**

5 Deployment Strategy **7**

6 Key Considerations & Risks **7**

7 Conclusion **8**

1 Introduction

This document outlines a comprehensive technical proposal for *Residency Companion*, an ambitious AI-powered medical information and decision-support tool tailored specifically for the Indian healthcare landscape. Inspired by the functionalities of *OpenEvidence*, *Residency Companion* aims to empower Indian physicians with rapid, evidence-based insights derived from the latest peer-reviewed medical literature and local guidelines.

The proposal details the understanding of the client's vision, an architectural blueprint, a phased development plan, and a robust technology stack designed for scalability, accuracy, and compliance.

2 Understanding the Vision: *Residency Companion* as an *OpenEvidence* Competitor

The core idea is to create a localized version of *OpenEvidence* for India. *OpenEvidence* serves as an AI-powered tool for physicians, facilitating access, search, synthesis, and application of medical literature at the point of care.

2.1 Key Functions of *OpenEvidence*

- ☑ **Natural Language Q&A:** Physicians ask questions in natural language, receiving evidence-grounded answers.
- ☑ **Evidence-Backed Search & Research:** Aggregates medical research, guidelines, and journals, allowing searchable access with citations.
- ☑ **Clinical Workflow Integration:** Supports drafting prior authorization letters, patient handouts, clinical calculators, diagnostic reasoning, and transcription of patient encounters.
- ☑ **Evidence Updates / Alerts:** Notifies doctors of new evidence related to previously inquired topics.
- ☑ **High Assurance, Trust & Compliance:** Emphasizes licensed sources, HIPAA compliance (U.S.), and physician verification.

2.2 How *OpenEvidence* Works (Technical Components)

Component	What it Must Do	How it is Likely Done
Content Ingestion	Collect full-text articles, guidelines from trusted sources; maintain content currency.	Licensing, document parsing (PDF/HTML), custom pipelines.
Search / Retrieval	Retrieve relevant documents/passages based on query; rank by relevance, recency, credibility.	Keyword search + embeddings/semantic retrieval (vector-search), filtering.
NL Understanding & Generation	Understand NL queries; generate answers, summaries; cite sources; surface pros/cons.	Large Language Models (LLMs), fine-tuned for medicine, RAG, summarization modules.
Verification & Credibility	Check source reliability; provide citations; indicate evidence strength; avoid outdated info.	Human curation, editorial oversight, internal scoring, clinician feedback.

Continued on next page

Table 1 – continued from previous page

Component	What it Must Do	How it is Likely Done
Clinical Workflow Tools	Transcription, templates, calculators, document generation.	Speech-to-text, template engines, coded calculators, document generation logic.
User Authentication	Ensure access only for verified healthcare professionals; manage permissions.	Credential verification (*NMC*, NPIs), secure login, privacy compliance.
Security, Privacy & Compliance	Secure data at rest/in transit; limit PHI storage; reliable infrastructure.	Encryption, secure cloud infrastructure, external audits, compliance frameworks.

2.3 Challenges Adaptations for India

Building an *OpenEvidence*-style competitor in India requires specific adaptations:

2.3.1 Local Content & Guidelines: Indian medical journals (e.g., IJMR, NMJI), *NMC* guidelines, ICMR, AIIMS, WHO/India region-specific guidelines, and resource-appropriate recommendations are crucial.

2.3.2 Credential Verification: Integration with existing Indian registration numbers (*NMC* / National Medical Commission) and hospital affiliations.

2.3.3 Technical Infrastructure: Data pipelines for Indian content, NLP/LLM models fine-tuned with Indian datasets, and secure, scalable hosting within India.

2.3.4 Feature Set: The core Q&A and evidence search functionalities remain central.

2.3.5 Problem-Solution Mapping for India:

- **Updating content / licensing:** Automate ingestion of open-access Indian journals; partner with publishers for institutional licensing.
- **Local guidelines mismatch:** Build a localized guideline engine from ICMR, *NMC*, AIIMS, WHO-SEARO.
- **Regulatory / legal compliance:** On-device processing, India-hosted cloud, compliance audits (*DPDP Act* Act).
- **Credential verification:** Integrate *NMC* registry API, hospital email verification, Aadhaar eKYC.
- **AI hallucinations/errors:** Use *Retrieval-Augmented Generation (RAG)* with mandatory inline citations, confidence scores, physician feedback.
- **Adoption & trust:** Peer review layer, CME accreditation, hospital pilots.

2.4 Sourcing Training Material (Indian Context)

A multi-tiered approach for data sourcing is planned:

- **Government & Guideline Documents:** (e.g., ICMR, NIHFW, NHP, NHM, AIIMS) - Completely open and authoritative.
- **Open Access Journals:** (e.g., IJMR, NMJI, CIJMR, IJMS, MedIND older volumes) - Safe for ingestion after license checks.

- **Publisher Content:** (e.g., JAPI new issues) - Requires licensing agreements.

Specific leads for open-access sources and potential APIs have been identified.

3 Development Plan: *Residency Companion* Technical Roadmap

The technical roadmap is structured into four distinct phases, prioritizing the client's immediate need for a content ingestion and management system, followed by the robust AI backend and multi-channel delivery.

3.1 Core Technology Choices

- **Backend Framework:** *FastAPI* (Python) for high performance and ease of development.
- **Primary AI Model:** *GPT-5 API* (OpenAI) for state-of-the-art Natural Language Understanding and Generation, with plans for fine-tuning.
- **Core AI Architecture:** *Retrieval-Augmented Generation (RAG)* to provide evidence-based answers with citations and minimize hallucinations.
- **Multi-Channel Delivery:** Web application, Mobile application, and WhatsApp integration.
- **Deployment Environment:** Cloud-based (AWS/GCP/Azure India regions for data locality and compliance).

3.2 Phase 1: Foundation & Content Ingestion System (MVP)

3.2.1 Objective: Establish core infrastructure and deliver a functional system for the client to feed and manage documents (PDFs, website links).

1.1. Setup & Core Infrastructure

- **Activities:** Project repositories, cloud account setup, managed PostgreSQL (with `pg_vector`), Optional: S3-compatible object storage, managed Redis/SQS for queueing, CI/CD pipeline setup.
- **Technologies:** Git, AWS/GCP/Azure, PostgreSQL (`pg_vector`), S3, Redis, Docker, GitHub Actions/GitLab CI.
- **Deliverables:** Initial cloud resources provisioned, CI/CD ready.

1.2. Client Dashboard (Frontend)

- **Activities:** UI/UX design, frontend development for login, PDF/URL upload forms, and a "Knowledge Base Management" page (list documents, status, metadata, re-process/delete).
- **Technologies:** React/Vue/Angular, Tailwind CSS/MUI.
- **Deliverables:** Functional client-facing web dashboard.

1.3. Backend API for Content Management (FastAPI)

- **Activities:** Database schema design (`documents`, `document_chunks`, `users`), FastAPI endpoints for document upload, URL submission, listing, retrieval, deletion, and reprocessing. Authentication/authorization for client access.
- **Technologies:** *FastAPI*, SQLAlchemy/Alembic, Pydantic, `boto3`, `redis-py`.
- **Deliverables:** *FastAPI* service with core content management endpoints.

1.4. Data Ingestion & Pre-processing Pipeline (Worker Services)

- **Activities:** Asynchronous workers consuming tasks from queue. PDF parsing (text extraction, basic table handling, metadata), Web scraping (main content extraction, cleaning, metadata). Both workers enqueue tasks for the next phase (Chunking & Embedding).
- **Technologies:** Python, `pdfminer.six`/`pypdf`, `BeautifulSoup4`, `requests`, `Scrapy`, `redis-queue`.
- **Deliverables:** Background worker services for PDF parsing and web scraping.

3.3 Phase 2: Knowledge Base Creation & AI Integration

3.3.1 Objective: Transform processed content into an AI-ready knowledge base and implement the core evidence-based Q&A system.

2.1. Text Chunking & Embedding Service

- **Activities:** Implement smart text chunking (sentence-aware, token limits). Integrate initial general-purpose sentence embedding model. Research and integrate/fine-tune medical-specific embedding models. Store chunks, embeddings, and metadata into the vector database.
- **Technologies:** Python, `sentence-transformers`, `NLTK`/`SpaCy`, `pg_vector` (or Pinecone/Weaviate).
- **Deliverables:** Microservice for text chunking and embedding, populated vector database.

2.2. AI-Powered Q&A Backend (FastAPI)

- **Activities:** Query embedding. *Retrieval-Augmented Generation (RAG)* Orchestrator: Hybrid search (semantic + keyword) over chunks. Context assembly. *GPT-5 API* integration (API calls, retry mechanisms, error handling). Response generation with clear bibliography/references by tracing source IDs.
- **Technologies:** *FastAPI*, `openai` Python client, `psycopg2`, `pg_vector`, LangChain/LlamaIndex (for RAG).
- **Deliverables:** Functional API endpoint for evidence-based Q&A with *GPT-5 API* and citations.

3.4 Phase 3: Multi-Channel User Interfaces & Enhancements

3.4.1 Objective: Deliver the AI capabilities to end-users across web, mobile, and WhatsApp, ensuring compliance and security.

3.1. Web Application (Physician-Facing Frontend)

- **Activities:** UI/UX design for Q&A, physician authentication (*NMC* API integration, secure login). Q&A interface, answer display with citations, feedback mechanism, basic user profile.
- **Technologies:** React/Vue/Angular, Auth0/Cognito (or custom auth).
- **Deliverables:** Public-facing web application for physicians.

3.2. Mobile Application (iOS/Android)

- **Activities:** Cross-platform development (Flutter), mobile-optimized Q&A interface, potential voice input, push notification integration (for future updates).
- **Technologies:** Flutter.
- **Deliverables:** Initial mobile app for iOS and Android.

3.3. WhatsApp Integration

- **Activities:** WhatsApp Business API setup, webhook listener (FastAPI endpoint), message processing and response formatting for WhatsApp (concise answers with citations, handling character limits), simple user verification.
- **Technologies:** *FastAPI*, WhatsApp Business API (Twilio/Cloud API).
- **Deliverables:** Functional WhatsApp bot for Q&A.

3.5 Phase 4: Advanced Features Optimization

3.5.1 Objective: Add advanced capabilities and continuously refine performance and accuracy.

4.1. Fine-tuning *GPT-5 API* (or Alternative LLM)

- **Activities:** Curate high-quality, task-specific datasets from ingested Indian medical content. Utilize OpenAI's fine-tuning APIs. Rigorous evaluation with medical experts.
- **Technologies:** OpenAI fine-tuning API, custom Python scripts for data, evaluation frameworks.
- **Deliverables:** Fine-tuned *GPT-5 API* model (or equivalent).

4.2. Evidence Updates / Alerts

- **Activities:** Implement change detection for new content relevant to past queries. Develop alerting logic. Integrate with email, push notifications, or WhatsApp.
- **Technologies:** *FastAPI*, background cron jobs, notification services.
- **Deliverables:** Automated evidence update alerts.

4.3. Clinical Workflow Tools (Future Iterations)

- **Activities:** Develop modular microservices for clinical calculators, explore LLM-driven diagnostic support (with extreme caution), and template-based document drafting (e.g., prior authorization).
- **Technologies:** *FastAPI*, specialized medical libraries, LLM integration.
- **Deliverables:** Modular clinical tools.

4 Technology Stack Summary

- **Backend:** *FastAPI* (Python), SQLAlchemy, Pydantic
- **Databases:** PostgreSQL (`pg_vector`), Redis
- **Object Storage:** AWS S3 / GCP Cloud Storage / Azure Blob Storage
- **Asynchronous Tasks:** Redis Queue / Celery (with Redis as broker)
- **Core AI:** *GPT-5 API* (OpenAI), `openai` Python client
- **Embedding Models:** `sentence-transformers`, Medical-specific BERT variants (fine-tuned)
- ***Retrieval-Augmented Generation (RAG)* Orchestration:** LangChain / LlamaIndex
- **Containerization:** Docker
- **Orchestration:** Kubernetes (EKS, GKE, AKS) **Cloud Platform:** AWS / GCP / Azure (India region preference)
- **Frontend (Client Dashboard & Web App):** React / Vue / Angular
- **Mobile App:** Flutter
- **Messaging:** WhatsApp Business API (via Twilio/Cloud API)
- **CI/CD:** GitHub Actions / GitLab CI
- **Monitoring/Logging:** Prometheus/Grafana, ELK Stack / CloudWatch / Stackdriver

5 Deployment Strategy

- **Containerization:** All services to be containerized using Docker.
- **Orchestration:** Kubernetes (EKS, GKE, AKS) for scalability, high availability, and efficient resource management.
- **Managed Services:** Leverage cloud-managed services for databases, queues, and object storage to reduce operational overhead.
- **Geographical Locality:** Primary deployment and data storage in Indian cloud regions for data residency and low latency.
- **API Gateways:** Utilize cloud API Gateways for secure and managed access to backend APIs.

6 Key Considerations & Risks

- ***GPT-5 API* Availability & Cost:** Need to factor in API access, usage costs, and fine-tuning expenses.
- **Data Quality:** Rigorous data validation and curation are paramount for the accuracy of responses.

- **Medical Accuracy & Hallucinations:** Implement robust *Retrieval-Augmented Generation (RAG)*, careful prompt engineering, and human medical oversight to mitigate inaccuracies.
- **Regulatory Compliance:** Continuous monitoring and adaptation to evolving Indian health data regulations (*DPDP Act* Act).
- **Physician Trust & Adoption:** Building trust through accuracy, transparency (citations), and peer validation is crucial.
- **Scalability:** Design the architecture to handle anticipated growth in data volume and user concurrency across multiple channels.

7 Conclusion

The *Residency Companion* project, leveraging modern AI techniques like *Retrieval-Augmented Generation (RAG)* with powerful LLMs like *GPT-5 API*, integrated into a robust *FastAPI* backend and delivered via multiple user channels, presents a transformative opportunity for medical practitioners in India. This plan provides a clear, phased approach to realize this vision, starting with immediate value delivery for content management and scaling towards a comprehensive, intelligent decision-support system.